

Python - Rekenmachine

D. Leeuw

2 juni 2026

v.0.1.0



© 2025-2026 Dennis Leeuw

Dit werk is uitgegeven onder de Creative Commons BY-NC-SA Licentie en laat anderen toe het werk te kopiëren, distribueren, vertonen, op te voeren, en om afgeleid materiaal te maken, zolang de auteurs en uitgever worden vermeld als maker van het werk, het werk niet commercieel gebruikt wordt en afgeleide werken onder identieke voorwaarden worden verspreid.

1 Python als rekenmachine

Programmeertalen kunnen ook gebruikt worden om te rekenen, daarbij kunnen ze de traditionele rekenmachine vervangen. Start de Python-interpreter en gebruik Python als simpele rekenmachine zoals dat hieronder staat:

```
Python 3.11.2 (main, Nov 30 2024, 21:22:50) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 6+7
13
>>> 7-6
1
>>> 6-7
-1
>>> 7*6
42
>>> 7/6
1.1666666666666667
>>>
```

We zien heel simpele zaken als optellen, aftrekken, vermenigvuldigen en delen.

Zo kunnen we ook machtsverheffen en worteltrekken:

```
Python 3.11.2 (main, Nov 30 2024, 21:22:50) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 5**2
25
>>> 25**(1/2)
5.0
>>>
```

We gebruiken hier `()` om het delen van $1/2$ voorrang te geven op het machtsverheffen. We kunnen met haakjes dus bepalen wat er voorrang heeft boven de normale rekenregels:

```
>>> 5+6*7+8
55
>>> (5+6)*(7+8)
165
```

Vermenigvuldigen gaat voor optellen. Bij de eerste som wordt dus eerst $6*7$ gedaan waarna er 5 en 8 bij opgeteld worden. Als we haakjes gebruiken worden eerst $5+6$ en $7+8$ uitgevoerd waarna de uitkomsten van de beide optellingen (11 en 15) met elkaar vermenigvuldigd worden.

Bij het worteltrekken zit er een kleine onnauwkeurigheid als we bijvoorbeeld met derdemachten werken, gelukkig komen die bij ons niet vaak voor:

```
>>> 5**3
125
>>> 125**(1/3)
4.999999999999999
```

De onnauwkeurigheid wordt veroorzaakt omdat $1/3$ geen mooi rond getal is en dus zelf al een onnauwkeurigheid heeft. Er zijn functies in Python om wel nauwkeurig te werken met wortels, maar daar gaan we hier niet nader op in.

1.1 Opdrachten

Gebruik Python om de volgende sommen uit te rekenen.

1. Joris is jarig en hij heeft 20 taartjes meegenomen. In zijn klas zitten 17 medeleeringen en er is een juf. Iedereen krijgt 1 taartje. Hoeveel taartjes houdt Joris over?
2. Ahmed viert Suikerfeest met zijn familie. Er komen 24 familieleden langs en Ahmed moet brood halen voor bij de maaltijd. Elk familielid eet ongeveer een half brood. Hoeveel brood moet Ahmed halen?

1.2 Verdieping modulo en floor division

Bij het rekenen hebben we ook nog de geheeltallige deling (floor division) en de bijkomende modulo- of klok-rekenen. Om deze begrippen uit te leggen beginnen we met een voorbeeld.

Stel we hebben bij het begin van een nieuw schooljaar 213 studenten die willen gaan studeren. Er kunnen uit praktische overwegingen 30 studenten in

een klas. Hoeveel klassen kunnen we dan volledig vullen en hoeveel studenten blijven er over?

Als we dit willen uitrekenen kunnen we het zo doen:

```
>>> 213/30
7.1
>>> 30*7
210
>>> 213-210
3
```

We hebben dus 7 klassen van 30 studenten en dus houden we 3 studenten over. Dit kan ook korter:

```
>>> 213//30
7
>>> 213%30
3
```

De dubbele slash `//` is de “floor division” ofwel die geeft weer hoeveel keer het volledige aantal in het totaal past. Met het `%` character berekenen we de modulo, kortom wat blijft er over na de vorige berekening.

Het voorgaande lijkt misschien een voorbeeld waar je niet zo vaak mee te maken krijgt, toch komen dit soort berekeningen redelijk vaak voor, wat duidelijk gemaakt wordt door de term klok-rekenen. Stieken doen we namelijk behoorlijk vaak aan modulo-rekenen, zonder dat we het weten. Als we horen dat we om 13:00 uur een afspraak hebben dan weten we dat dat om 1 uur is. Eigenlijk is onze “groep”-grote 12 en trekken we 12 van 13 af om op 1 uur uit te komen. Zo ook bij 16:00 uur waar we 12 van 16 aftrekken om op 4 uur uit te komen. Omdat we er altijd alleen maar 12 vanaf hoeven te trekken als het om de klok gaat valt het niet direct op, dat de echte wiskundige berekening feitelijk is:

```
>>> 16/12
1.3333333333333333
>>> 12*1
12
>>> 16-12
4
```

Om nog een stapje verder te gaan, het volgende voorbeeld. Stel we weten dat een auto er 6455768 seconden over doet om een bepaalde afstand af te leggen. Hoeveel uur, minuten en seconden heeft auto er dan over gedaan. Dat kunnen we berekenen met floor division en modulo-rekenen. We weten dat er 60 seconden in een minuut zitten en 3600 seconden in een uur. We gaan eerst de uren breken:

```
>>> 6455768//3600
1793
>>> 6455768%3600
968
```

We hebben dus 1793 uur gereden en er zijn 968 seconden over. Nu gaan we uitzoeken hoeveel minuten dat is en wat er dan nog aan seconden over blijft:

```
>>> 968//60
16
>>> 968%60
8
```

Er zijn dus 16 minuten verstreken en 8 seconden over. Het totaal komt dan uit op 1793 uur, 16 minuten en 8 seconden.

1.2.1 Opdrachten

1. Pieter heeft een duurloop gedaan en hij heeft daar 145476 seconden over gedaan. Hoeveel dagen, uren, minuten en seconden heeft Pieter hardgelopen?

Index

aftrekken, 2

delen, 2

machtsverheffen, 2

optellen, 2

rekenen, 2
 aftrekken, 2

delen, 2

machtsverheffen, 2

optellen, 2

vermenigvuldigen, 2

worteltrekken, 2

vermenigvuldigen, 2

worteltrekken, 2